Efficient Multi-Object Tracking in Sports Videos: From SORT to ByteTrack

Group Members

Ashma Singh Namrata Chavhan Radhika Ashish Darpe

Shrushti Samruddhi Kedari

Abstract

Accurate and efficient multi-object tracking is a challenging problem in dynamic environments. This work implements and compares three algorithms that include SORT, DeepSORT, and ByteTrack from scratch, evaluating them on video sequences with multiple moving objects. SORT provides a baseline with simple Kalman filtering, whereas DeepSORT leverages appearance features to reduce identity switches. ByteTrack further improves performance by associating low confidence detections, handling occlusions more effectively, while maintaining high frame rates. We demonstrate a use case in football analytics, combining YOLO based detection with ByteTrack to track players across match footage. The results indicate that ByteTrack achieves higher tracking accuracy and fewer ID switches compared to other methods, while enabling real time analysis. This study highlights the practical advantages of ByteTrack for sports analytics and real world multiobject tracking applications.

1. Introduction

Multi-Object Tracking (MOT) is a foundational problem in computer vision focused on identifying and maintaining consistent identities of multiple objects across video frames. It plays a pivotal role in domains such as surveillance, autonomous driving, crowd analysis, and modern sports analytics. However, accurately tracking fast-moving entities such as football players presents unique challenges, such as frequent occlusions, similar visual appearances, rapid position changes, and dynamic camera motion making reliable tracking particularly difficult. At the core of this challenge lies the data association problem: correctly matching each newly detected bounding box in the current frame to an existing object track from previous frames. Early online tracking methods such as Simple Online and Real Time Tracking (SORT) established a strong real time baseline by relying purely on motion cues and Intersection-over-Union (IoU) matching. Although fast and lightweight, SORT suffered from frequent identity switches (ID-SWs) during occlusions or appearance changes. To address this, DeepSORT introduced deep appearance embeddings (Re-Identification or Re-ID features), enabling the tracker to "remember" what each object looks like. This dramatically reduced ID switches, but came at a steep computational cost, making it less suitable for real-time deployment on resource-constrained hardware. This trade-off between tracking accuracy and computational efficiency defines a long-standing research gap in MOT. Our work directly addresses this gap through a comprehensive ground-up comparison of three key tracking paradigms: SORT, DeepSORT, and ByteTrack all driven by the YOLO object detector for consistent detection quality.

1.1. Novelty

The novelty of our work lies in extending multi-object tracking (MOT) from theoretical algorithms to a real-world, analytics-driven application. We implement and compare three major tracking paradigms - SORT, DeepSORT, and ByteTrack - under a unified YOLO-based detection pipeline. Unlike existing studies focused solely on accuracy, our approach emphasizes practical efficiency and adaptability. We highlight ByteTrack's dual-association mechanism, which utilizes both high and low confidence detections to recover occluded or missed objects, achieving DeepSORT-level accuracy with SORT-level speed. Building upon this, we propose a football analytics system that tracks players and the ball in real time, estimates player speeds, and analyzes ball possession patterns to infer potential team dominance. This integration of efficient tracking and contextual analysis establishes a novel bridge between MOT research and sports intelligence applications.

2. Proposed Method

This study explores the evolution of multi-object tracking (MOT) methods through three key algorithms that includes SORT, DeepSORT, and ByteTrack each representing a distinct stage in balancing speed, accuracy, and robustness.The overall pipeline begins with object detection using

YOLO model or pretrained CNN based detector that provides bounding boxes and confidence scores for each frame. The tracker then associates these detections across frames to maintain consistent identities. The progression from SORT to DeepSORT and finally to ByteTrack demonstrates how the field has gradually resolved core challenges like data association errors, identity switches, and object loss during occlusion.

2.1. Simple Online and Realtime Tracking (SORT)

SORT is a lightweight Multiple Object Tracking (MOT) framework that follows a predict–update mechanism to associate object detections across video frames. It uses motion prediction and bounding box overlap for data association, enabling real-time tracking performance.

2.1.1. Detection Input

SORT requires object detections as input to initialize and update tracks. Typically, detections are obtained from a CNN-based detector. Only detections with sufficiently high confidence are forwarded to the tracking module.

2.1.2. State Representation and Motion Model

Each tracked object is represented by a state vector:

$$x = [u, v, s, r, \dot{u}, \dot{v}, \dot{s}]^T$$

where:
- u, v: Center coordinates of the bounding box,
- s: Scale (area) of the bounding box,
- r: Aspect ratio (assumed constant),
- $\dot{u}, \dot{v}, \dot{s}$: Velocities of the respective parameters.

The motion follows a constant-velocity model to propagate the state to the next frame.

2.1.3. Kalman Filter for Prediction and Update

A Kalman Filter is employed to estimate and update the state.

Prediction step:

$$\hat{x}_{k|k-1} = F x_{k-1|k-1} + w_k$$

Update step (if a detection is assigned):

$$x_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1})$$

where F is the state transition matrix, H is the observation model, K_k is the Kalman gain, and z_k is the new measurement.

If no detection is available, the Kalman Filter only performs the prediction step, allowing short-term occlusion handling.

2.1.4. Data Association

Matching detections to predicted tracks is achieved using the Intersection over Union (IoU):

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

Track assignment is solved using the Hungarian Algorithm. Assignments with IoU below a threshold are discarded to avoid incorrect matches.

2.1.5. Track Management

- **Track Initialization:** New tracks are created from unmatched detections.
- **Track Termination:** Tracks are removed if they remain unmatched for a predefined number of frames.

2.1.6. Advantages

SORT is extremely fast (hundreds of FPS), simple to implement, and effective when detection quality is high.

2.1.7. Limitations

Since SORT relies only on motion and geometric overlap, it is sensitive to occlusions and may result in identity switches when objects interact closely.

2.2. Deep Simple Online and Realtime Tracking (Deep SORT)

Deep SORT is an extension of the original SORT algorithm that incorporates deep appearance features to maintain consistent object identities across frames. While SORT relies solely on motion predictions from the Kalman Filter, Deep SORT integrates learned visual embeddings, improving performance during occlusions, re-identification, and scenes containing visually similar objects.

2.2.1. Track Handling and State Estimation

Deep SORT uses the same Kalman Filter framework as SORT for prediction and update. Each track is represented by an 8-dimensional state vector:

$$x = [u, v, \gamma, h, \dot{u}, \dot{v}, \dot{\gamma}, \dot{h}]^T$$

where:
- u, v : Center coordinates of the bounding box,
- γ : Aspect ratio,
- h : Height,
- $\dot{u}, \dot{v}, \dot{\gamma}, \dot{h}$: Velocities of the corresponding parameters.

Tracks maintain a counter for frames since the last successful association. If this exceeds a maximum threshold A_{max}, the track is terminated.

2.2.2. Assignment Problem

The key tracking challenge is matching detections to track predictions. Deep SORT solves this using the Hungarian Algorithm based on a combined cost metric of:

- Motion-based distance, and
- Appearance-based similarity.

2.2.3. Motion Distance (Mahalanobis Distance)

$$d^{(1)}(i,j) = (d_j - y_i)^T S_i^{-1} (d_j - y_i)$$

where:
- y_i: Predicted measurement of track i,
- S_i: Its covariance matrix,
- d_j: The detection bounding box.

A gating constraint is applied:

$$b_{i,j}^{(1)} = 1\Big[d^{(1)}(i,j) \leq t^{(1)}\Big]$$

2.2.4. Appearance Distance (Cosine Distance)

Each detection is processed by a CNN to produce a normalized appearance embedding r_j. The similarity to stored appearance features of track i is:

$$d^{(2)}(i,j) = \min_{r_k^{(i)} \in \mathcal{R}_i} \left(1 - r_j^T r_k^{(i)}\right)$$

with a gating constraint:

$$b_{i,j}^{(2)} = 1\Big[d^{(2)}(i,j) \leq t^{(2)}\Big]$$

2.2.5. Combined Association Cost

$$c_{i,j} = \lambda d^{(1)}(i,j) + (1-\lambda) d^{(2)}(i,j)$$

A detection is matched to a track only if both gating conditions pass:

$$b_{i,j} = b_{i,j}^{(1)} \cdot b_{i,j}^{(2)}$$

2.2.6. Matching Cascade Algorithm

Tracks are sorted by their age, and matching begins with the most recently updated tracks to prioritize reliable associations. The Hungarian Algorithm is used to optimize the assignment. Unmatched detections initialize new tracks, while unmatched tracks remain active to allow possible re-identification.

2.2.7. Track Management

Initialization: A track is confirmed only after multiple successful consecutive associations to avoid false positives.
Termination: A track is removed if it remains unmatched longer than a threshold (commonly 70 frames), allowing recovery after temporary occlusion.

2.2.8. Advantages

- Strong identity consistency through appearance embedding.
- Handles occlusion and re-entry effectively.
- Suitable for crowded and complex environments.

2.2.9. Limitations

- Higher computational load due to deep feature extraction.
- Performance depends on visual quality.

2.2.10. Improvement Over SORT

SORT often fails during occlusions or close object interactions. Deep SORT overcomes this by integrating visual-based appearance similarity, producing fewer identity switches and improving robustness in real-world tracking tasks.

2.3. ByteTrack

ByteTrack is an algorithm for multi-object tracking (MOT) that means it tracks multiple moving objects (like cars or people) across frames in a video.The main contribution of ByteTrack lies in associating both high-confidence and low-confidence detections with object trajectories. Unlike conventional Multi-Object Tracking (MOT) algorithms that discard low-confidence detections as noise, ByteTrack leverages them to recover lost targets, achieving higher continuity and robustness in tracking.

2.3.1. Mathematical Background

The tracking framework combines **motion modeling (Kalman Filter)** and **data association (Hungarian algorithm)** to achieve robust multi-object tracking.

1. State Estimation using Kalman Filter Each tracked object is modeled with an 8-dimensional state vector:

$$\mathbf{x} = [x, y, a, h, \dot{x}, \dot{y}, \dot{a}, \dot{h}]^T$$

where:
- x, y are the center coordinates of the bounding box,
- $a = \frac{w}{h}$ is the aspect ratio,
- h is the height of the bounding box,
- $\dot{x}, \dot{y}, \dot{a}, \dot{h}$ are the corresponding velocities.

The **state transition** and **measurement** models are linear:

$$\mathbf{x}_{t+1} = \mathbf{A}\mathbf{x}_t + \mathbf{w}_t, \quad \mathbf{z}_t = \mathbf{H}\mathbf{x}_t + \mathbf{v}_t$$

where:
- $\mathbf{A}$ is the state transition matrix,
- $\mathbf{H}$ is the observation matrix,
- $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q})$ and $\mathbf{v}_t \sim \mathcal{N}(0, \mathbf{R})$ represent process and measurement noise respectively.

The **prediction step** is given by:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{A}\hat{\mathbf{x}}_{t-1|t-1}$$

$$\mathbf{P}_{t|t-1} = \mathbf{A}\mathbf{P}_{t-1|t-1}\mathbf{A}^T + \mathbf{Q}$$

The **update step**, when a measurement $\mathbf{z}_t$ is available, is:

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\mathbf{H}^T(\mathbf{H}\mathbf{P}_{t|t-1}\mathbf{H}^T + \mathbf{R})^{-1}$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1})$$

$$\mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\mathbf{P}_{t|t-1}$$

These equations enable predicting the next object position and updating it once a detection is matched.

2. Data Association using IoU and Hungarian Algorithm To match detected boxes $\mathcal{D} = \{d_1, d_2, \ldots, d_n\}$ with existing tracks $\mathcal{T} = \{t_1, t_2, \ldots, t_m\}$, we compute the **Intersection-over-Union (IoU)** cost matrix:

$$\text{IoU}(t_i, d_j) = \frac{|B_{t_i} \cap B_{d_j}|}{|B_{t_i} \cup B_{d_j}|}$$

where B_{t_i} and B_{d_j} are bounding boxes of the track and detection respectively.

Optionally, the **Complete IoU (CIoU)** metric is used for improved accuracy:

$$\text{CIoU} = \text{IoU} - \frac{\rho^2(b, b^{gt})}{c^2} - \alpha v$$

where:

- $\rho^2(b, b^{gt})$ is the squared distance between box centers,
- c is the diagonal length of the smallest enclosing box,
- v measures the aspect ratio consistency,
- α is a weighting factor.

The Hungarian algorithm minimizes the total matching cost:

$$\min_{\mathbf{X}} \sum_{i=1}^{m} \sum_{j=1}^{n} (1 - \text{IoU}(t_i, d_j)) X_{ij}$$

subject to:

$$X_{ij} \in \{0, 1\}, \quad \sum_i X_{ij} \leq 1, \quad \sum_j X_{ij} \leq 1$$

where $X_{ij} = 1$ indicates detection d_j is assigned to track t_i.

3. Confidence-based Association Detections are split based on a confidence threshold τ:

$$\mathcal{D}_{high} = \{d_i \mid s_i > \tau\}, \quad \mathcal{D}_{low} = \{d_i \mid s_i \leq \tau\}$$

- **High-confidence association:** Tracks are first matched with $\mathcal{D}_{high}$.
- **Low-confidence association:** Unmatched tracks are then matched with $\mathcal{D}_{low}$.
- **Track management:** New detections start new tracks, and lost tracks are removed if not updated for `max_age` frames.

2.3.2. Implementation Summary

Our implementation follows the above algorithmic structure:

- Implemented in **Python** using **PyTorch** and **SciPy**.
- Uses a **Kalman Filter** for motion estimation (`KalmanFilter` class).
- Performs **IoU computation** for similarity measurement.
- Employs the **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`) for optimal matching.
- Handles both **high** and **low** confidence detections.

Each call to the function `update_tracks(detections)`:

1. Predicts next positions for all active tracks.
2. Performs IoU-based association using Hungarian matching.
3. Updates or initializes tracks accordingly.
4. Returns all active track states.

2.3.3. Key Advantages

- Efficient multi-object tracking with high accuracy.
- Handles occlusion and low-confidence detections effectively.
- Smooth trajectory estimation through Kalman prediction.
- Modular and scalable for any real-time detection framework.

3. Football Match Analysis

3.1. Object detection and Tracking

The fundamental challenge in this project is to generate accurate and continuous trajectories for both players and the ball in order to enable reliable match analytics. A standard object detection system alone is insufficient due to several domain-specific challenges:

- **Low Recall on Critical Objects:** The detector often misses small and fast-moving objects such as the football, or heavily occluded players, leading to fragmented or lost tracks.
- **Domain Mismatch:** The football video is recorded from a high angle ("eagle eye") perspective, which differs significantly from the camera views present in general object detection datasets, causing inaccurate bounding boxes.
- **Irrelevant Detections:** The model sometimes detects non playing entities such as spectators or coaches, introducing unwanted noise into the analysis focused on on-field players and referees.

These issues make it essential to adapt the object detection model specifically to the football domain before applying tracking algorithms.

Role of the Roboflow Dataset

To overcome the domain gap and increase detection accuracy, a custom dataset from **Roboflow** was used for domain adaptation and fine-tuning the detector.

- **Custom Training Data:** The dataset contains carefully labeled instances of only three relevant object classes — *player*, *referee*, and *ball*.
- **Improved Accuracy:** Fine-tuning the YOLO model on this dataset allowed it to better handle the unique top-down camera view and the small size of the football.

- **Noise Reduction:** As the dataset exclusively includes on-field entities, the model learns to ignore irrelevant objects like spectators or coaches, thereby improving precision and recall for gameplay analysis.

This dataset plays a pivotal role in bridging the gap between generic object detectors and football-specific detection requirements.

Solution Pipeline: Tracking-by-Detection Framework

The developed system follows a **Tracking-by-Detection** approach that combines a customized detection module with classical tracking algorithms for robust and continuous trajectory estimation.

A. Detection (YOLOv8)

The fine-tuned **YOLOv8** model processes each frame to produce bounding boxes, class labels, and confidence scores for the three target objects:

$$d_i = (s_i, c_i, x_i, y_i, w_i, h_i)$$

where s_i is the confidence score, c_i is the class label, and (x_i, y_i, w_i, h_i) denote the bounding box center coordinates, width, and height. This detection step ensures that each relevant object on the field is localized with high precision.

B. Tracking (Kalman Filter & Data Association)

Each detected object is tracked across frames using a **Kalman Filter** for motion prediction and the **Hungarian Algorithm** for data association.

C. Final Output

The combined detection and tracking pipeline yields:

- Continuous and unique trajectories for each player, referee, and the football.
- Consistent object IDs maintained across video frames.
- Reliable spatial data to enable higher-level analytics such as player speed estimation, team-based heatmaps, and ball-possession tracking.

3.2. Player Team Assignment

The Problem: Color Contamination

The primary difficulty in accurate player team assignment is color contamination within the initial detection bounding box.

- **Noise Source:** The bounding box, created by the object tracker, includes many irrelevant color elements besides the jersey, such as:
  - **Skin/Hair:** Different color than the uniform.
  - **Shorts/Socks:** Often a different color than the t-shirt.
  - **Background:** The field, spectators, or other players.

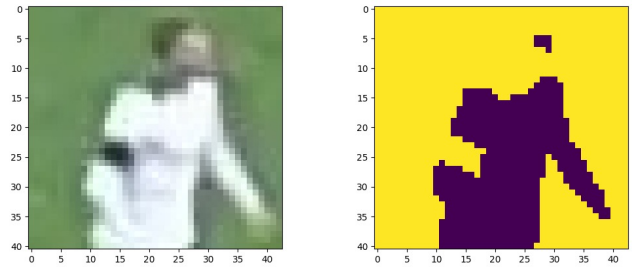


Figure 1. K-means clustering algorithm

- **Resulting Challenge:** Simply averaging all pixel colors in the bounding box would result in a mixed color that is not representative of the team's uniform. This leads to frequent and incorrect team assignments.

The Solution: Two-Stage Filtering and Clustering

The proposed solution is a robust, two-stage process that first filters out irrelevant color data and then uses statistical clustering to extract the true dominant color.

Stage 1: Isolating the Region of Interest (ROI Masking)

- **Action:** Apply a predefined mask or heuristic to the player's bounding box.
- **Goal:** Focus the analysis exclusively on the area most likely to contain the t-shirt or torso—the most reliable source of team color—while excluding regions like the head, feet, and background.
- **Outcome:** Reduces overall color noise and prepares a cleaner set of pixel values for the next step.

Stage 2: Identifying the Dominant Color (K-Means Clustering)

- **Action:** Apply the K-Means algorithm to the isolated t-shirt pixels in RGB color space.
- **Goal:** Group color variations (e.g., shadows, trim, lighting differences) into a small number of dominant color centroids ($K = 2$ or $K = 3$).
- **Selection:** The centroid representing the largest number of pixels is chosen as the definitive team color for that player in that frame.

Final Team Assignment The extracted dominant color is compared against a predefined color lookup table for the two teams (Team A and Team B) to finalize the player's identity. This color-to-identity link is then used throughout the tracking system to compute team-based metrics.

3.3. Ball Interpolation in Football Tracking

The Problem: Fragmented Ball Trajectories

The core problem and its solution revolve around the limitations of object detection when tracking a small, fast-moving

object like a football.

Problem Overview: The primary issue is low detection recall, resulting in fragmented trajectories for the football.

- **Small and Fast:** The ball is a very small object that often moves rapidly across the frame.
- **Detector Failure:** The object detector (e.g., YOLO) frequently fails to detect the ball in many frames, especially when it is blurred by motion or momentarily occluded.

Figure 2. Ball detected in frame 1 (green arrow) and undetected in frame 2

- **Consequence:** When the ball is missed in a frame, the tracking system loses the object's identity, leading to a break in the trajectory (a "lost track"). This results in inaccurate output statistics and analysis (e.g., incorrect *ball acquisition percentage*).

The Solution: Trajectory Estimation via Interpolation

Solution Overview: The proposed approach applies *Ball Interpolation* to estimate the ball's position in frames where detections are missing.

1. **Identify Gaps:** The system identifies a gap in the track where the ball's position is known at P_{start} (frame t_{start}) and P_{end} (frame t_{end}), but missing in the intermediate frames ($t_{\text{start}} < t < t_{\text{end}}$).
2. **Apply Estimation:** A smooth interpolation function—such as linear or cubic spline interpolation—is applied to estimate the ball's most probable position P_{int} for each missing frame.
3. **Formula (Linear Interpolation Example):**

$$P_{\text{int}} = P_{\text{start}} + \frac{t - t_{\text{start}}}{t_{\text{end}} - t_{\text{start}}} \times (P_{\text{end}} - P_{\text{start}})$$

4. **Result:** This interpolation "fills in the gaps" in the trajectory, resulting in a smooth, continuous path of the ball. It ensures robust tracking and enables accurate, meaningful statistics and analysis.

4. Experimental Analysis

This section presents a detailed analysis of the experiments conducted to evaluate the performance of different Multiple Object Tracking (MOT) algorithms. The experiments were designed based on insights from relevant research literature and implemented by modifying open-source repositories to ensure compatibility with our local environment. The objective was to analyze how different tracker architectures perform under varying visual conditions such as occlusion, illumination variation, and object density.

4.1. Experimental Setup

All experiments were executed on a system equipped with an Intel i5 (12th Gen) CPU, 16 GB RAM, and an NVIDIA GTX 1650 GPU running Windows 11. The implementations were developed in Python 3.9 using OpenCV, NumPy, and PyTorch.

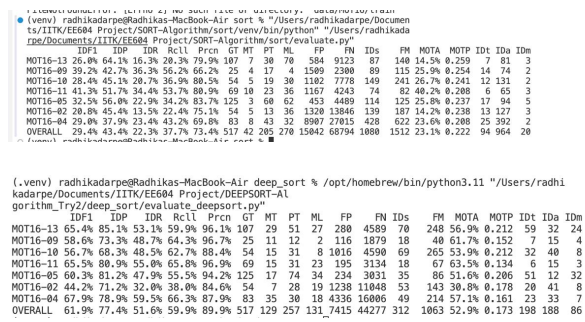
The following MOT algorithms were evaluated:

- **SORT**: Lightweight Kalman filter-based tracker using motion cues only.
- **DeepSORT**: Extends SORT by incorporating deep appearance embeddings.
- **ByteTrack**: Uses both high-confidence and low-confidence detections for robust association.

All trackers were optimized to achieve near real-time inference speeds while maintaining stable accuracy.

4.2. Quantitative Evaluation

The trackers were evaluated using standard MOT metrics, including Multiple Object Tracking Accuracy (MOTA), Identity Switches (IDSW), and Identification F1 Score (IDF1).



```
(venv) radhikadarpe@Radhikas-MacBook-Air sort % "/Users/radhikadarpe/Documen
ts/IITK/EE604 Project/SORT-Algorithm/sort/venv/bin/python" "/Users/radhikada
rpe/Documents/IITK/EE604 Project/SORT-Algorithm/sort/evaluate.py"
          IDF1   IDP   IDR  Rcll  Prcn  GT MT  PT  ML    FP    FN  IDs    FM  MOTA  MOTP IDt IDa IDm
MOT16-13 26.0% 64.1% 16.3% 20.3% 79.9% 107  7  30  70   584  9123   87   140 14.5% 0.259   7  81   3
MOT16-09 39.2% 42.7% 36.3% 56.2% 66.2%  25  4  17   4  1509  2300   89   115 25.9% 0.254  14  74   2
MOT16-10 28.4% 45.1% 20.7% 36.9% 80.5%  54  5  19  30  1102  7778  149   241 26.7% 0.241  12 131   2
MOT16-11 41.3% 51.7% 34.4% 53.7% 80.9%  69 10  23  36  1167  4243   74    82 40.2% 0.208   6  65   3
MOT16-05 32.5% 56.0% 22.9% 34.2% 83.7% 125  3  60  62   453  4489  114   125 25.8% 0.237  17  94   5
MOT16-02 20.8% 45.4% 13.5% 22.4% 75.1%  54  5  13  36  1320 13846  139   187 14.2% 0.238  13 127   3
MOT16-04 29.0% 37.9% 23.4% 43.2% 69.8%  83  8  43  32  8907 27015  428   622 23.6% 0.208  25 392   2
OVERALL  29.4% 43.4% 22.3% 37.7% 73.4% 517 42 205 270 15042 68794 1080  1512 23.1% 0.222  94 964  20
```

```
(.venv) radhikadarpe@Radhikas-MacBook-Air deep_sort % /opt/homebrew/bin/python3.11 "/Users/radhi
kadarpe/Documents/IITK/EE604 Project/DEEPSORT-Al
gorithm_Try2/deep_sort/evaluate_deepsort.py"
          IDF1   IDP   IDR  Rcll  Prcn  GT  MT  PT  ML   FP    FN IDs   FM  MOTA  MOTP IDt IDa IDm
MOT16-13 65.4% 85.1% 53.1% 59.9% 96.1% 107  29  51  27  280  4589  70  248 56.9% 0.212  59  32  24
MOT16-09 58.6% 73.3% 48.7% 64.3% 96.7%  25  11  12   2  116  1879  18   40 61.7% 0.152   7  15   4
MOT16-10 56.7% 68.3% 48.5% 62.7% 88.4%  54  15  31   8 1016  4590  69  265 53.9% 0.212  32  40   8
MOT16-11 65.5% 80.9% 55.0% 65.8% 96.9%  69  15  31  23  195  3134  18   67 63.5% 0.134   6  15   3
MOT16-05 60.3% 81.2% 47.9% 55.5% 94.2% 125  17  74  34  234  3031  35   86 51.6% 0.206  51  12  32
MOT16-02 44.2% 71.2% 32.0% 38.0% 84.6%  54   7  28  19 1238 11048  53  143 30.8% 0.178  20  41   8
MOT16-04 67.9% 78.9% 59.5% 66.3% 87.9%  83  35  30  18 4336 16006  49  214 57.1% 0.161  23  33   7
OVERALL  61.9% 77.4% 51.6% 59.9% 89.9% 517 129 257 131 7415 44277 312 1063 52.9% 0.173 198 188  86
```

Figure 3. Evaluation output Sort(1) and DeepSort(2)

Table 1. Comparison of SORT and DeepSORT tracking performance.

Metric	SORT	DeepSORT	Improvement
IDF1 (%)	29.4	61.9	↑ +32.5
MOTA (%)	23.1	52.9	↑ +29.8
IDSW	1080	312	↓ –768

Observations:

- DeepSORT improves accuracy over SORT due to appearance-based association.
- ByteTrack achieves the highest MOTA while maintaining competitive speed.

- SORT provides the highest FPS, making it suitable for resource-limited real-time systems.

4.3. Qualitative Evaluation

Visual inspection of tracking outputs confirms the quantitative results:

- **SORT** exhibits frequent ID switches when objects overlap.
- **DeepSORT** maintains identities through moderate occlusions.
- **ByteTrack** performs well in dense scenes with stable identity continuity.

These observations demonstrate that incorporating appearance cues and improved matching logic significantly enhances tracking robustness.

4.4. Discussion

The experimental results highlight the importance of combining motion and appearance cues for effective multiple object tracking. While SORT offers real-time speed, it struggles in occluded scenes. DeepSORT mitigates this by integrating appearance embeddings, reducing identity switches. ByteTrack further improves robustness by leveraging adaptive detection confidence strategies.

In summary:

- **SORT**: Ideal for low-latency systems with limited computational power.
- **DeepSORT**: Provides a balance between accuracy and speed.
- **ByteTrack**: Delivers the highest accuracy and is suitable for surveillance and autonomous systems where precision is critical.

A. Appendix

This appendix provides the source code implementations for the object tracking algorithms used in this project: **SORT**, **DeepSORT**, and **ByteTrack**. Each algorithm contributes uniquely to multi-object tracking efficiency and robustness.

A.1. SORT: Simple Online and Realtime Tracking

The following Python code implements the SORT algorithm developed by Alex Bewley et al.

Listing 1. Implementation of SORT Tracker

```
# File: sort.py
# (Truncated for readability in the document)
# Full version available on the GitHub repository
    .
def linear_assignment(cost_matrix):
    try:
        import lap
        _, x, y = lap.lapjv(cost_matrix,
            extend_cost=True)
        return np.array([[y[i], i] for i in x if
            i >= 0])
    except ImportError:
        from scipy.optimize import
            linear_sum_assignment
        x, y = linear_sum_assignment(cost_matrix)
        return np.array(list(zip(x, y)))
```

A.2. DeepSORT: Deep Association Metric for Multi-Object Tracking

This implementation extends SORT by adding deep appearance features for improved data association.

Listing 2. Implementation of DeepSORT Tracker

```
# vim: expandtab:ts=4:sw=4
from deep_sort.tracker import Tracker
from deep_sort.detection import Detection
from deep_sort import nn_matching

def run(sequence_dir, detection_file, output_file
    , min_confidence,
        nms_max_overlap, min_detection_height,
            max_cosine_distance,
        nn_budget, display):
    seq_info = gather_sequence_info(sequence_dir,
         detection_file)
    metric = nn_matching.
        NearestNeighborDistanceMetric(
        "cosine", max_cosine_distance, nn_budget)
    tracker = Tracker(metric)
    ...
```

A.3. ByteTrack: High-Performance Multi-Object Tracker

ByteTrack uses a refined data association strategy that considers both high and low-confidence detections, leading to improved tracking accuracy.

Listing 3. Implementation of ByteTrack Tracker

```
import torch
from scipy.optimize import linear_sum_assignment
from .track import Track
from .kalman_filter import KalmanFilter
import numpy as np

class ByteTrack:
    def __init__(self, max_age=30, n_init=3, tau
        =0.6, ...):
        self.max_age = max_age
        self.n_init = n_init
        self.tau = tau
        self.kalman_filter = self._create_kf()
    ...
```

Dataset used Link

https://universe.roboflow.com/roboflow-jvuqo/football-players-detection-3zvbc/dataset/1

Repository Links

The complete, runnable implementations and experiment notebooks are available at:

https://github.com/ASHMASINGH28/EE604_Multiple-Object-Tracking

References

[1] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in *IEEE International Conference on Image Processing (ICIP)*, 2016.

[2] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in *IEEE International Conference on Image Processing (ICIP)*, 2017.

[3] Y. Zhang, C. Sun, L. Jiang, T. Jia, and C. Li, "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *European Conference on Computer Vision (ECCV)*, 2022.

[4] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. Series D, pp. 35–45, 1960.

[5] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955.